Catalogue Quality Challenge Project Report

DSC PJATK Catalogue Quality Challenge
November 2026

1 Problem Description

The project aims to identify product offers that contain a controlled catalogue error. We built a model that checks whether the text and structured attributes describe a consistent offer. The final model achieved a mean validation ROC-AUC of 0.9924 on held-out sellers. This is a development result from the labelled training data; it is not a Kaggle leaderboard score.

Why catalogue consistency matters

A listing can contain several correct values that become suspicious when considered together. For example, a description may state that a package contains one main item while the structured pack size says five. A price or shipping weight may also look reasonable across the whole catalogue but be unusual for the particular product and quantity. These relationships make the task different from simply finding the largest or smallest numbers.

The model should also accept legitimate variation. A product may belong to more than one category, a promotion may explain a lower price, and some fields may be missing. We therefore learn normal catalogue patterns from the training data and measure how far each offer departs from them.

Prediction target

Each training offer has a binary target. A value of 0 means that no controlled corruption was applied; a value of 1 means that one controlled corruption was applied. A target of 0 does not guarantee perfect or complete real-world information. Missing values and unusual prices alone are not labels.

The required output is a probability between 0 and 1 for each unseen offer. Higher values mean greater estimated likelihood of controlled corruption. The submission contains the offer ID and this probability in two columns named id and target.

Evaluation goal

The main measure is ROC-AUC, which evaluates whether corrupted offers tend to receive higher scores than offers without controlled corruption. We also report Average Precision because corrupted offers are the minority class. Both measures evaluate model performance without choosing a fixed decision threshold. Neither should be read as classification accuracy.

2 Data

The labelled dataset contains 16,000 offers from 800 sellers. Each seller has 20 offers. There are 2,721 positive examples and 13,279 negative examples, so the positive fraction is about 17.0%. Predicting the negative class for every offer would therefore produce about 83% accuracy while detecting no corrupted offers.

Field	Meaning
id and seller_id	Offer identifier and seller group
title	Short product text
description	Longer product text and package information
declared_category	Category supplied with the offer
brand and condition	Brand and item condition
price_pln	Package price in Polish zloty
shipping_weight_kg	Shipping weight in kilograms
pack_size	Declared number of main items
target	Controlled corruption label, available only for training

The training data contains nine observed categories, five observed brands and three conditions: new, used and refurbished. Several text templates describe the same product using different names. The descriptions contain 180 distinct product phrases; these phrases include aliases and should not be interpreted as 180 independently verified product types.

Numeric values

Training attribute	Median	Observed minimum	Observed maximum
Price in PLN	77.51	0.97	3,464.27
Weight in kg	0.459	0.024	32.831
Pack size	1	1	5

Observed prices, weights and pack sizes are positive. Their ranges are wide because the catalogue contains different products and quantities. Missing values are retained as unknown observations, rather than being removed together with their offers.

Separation of analysis and inference

All descriptive analysis and model development use train.csv. The prediction file and sample submission are loaded only after training and validation are complete, for schema checks, ID alignment and inference. The final exported CSV contains 10,000 predictions. No distribution statistics from the prediction file are used to fit features or revise the model.

3 Exploratory Data Analysis

The EDA checks data integrity and looks for relationships that can support reliable features. Every table, plot and observation in this section comes from the training data. IDs are unique, the target is binary, and observed numeric values satisfy the expected positive ranges.

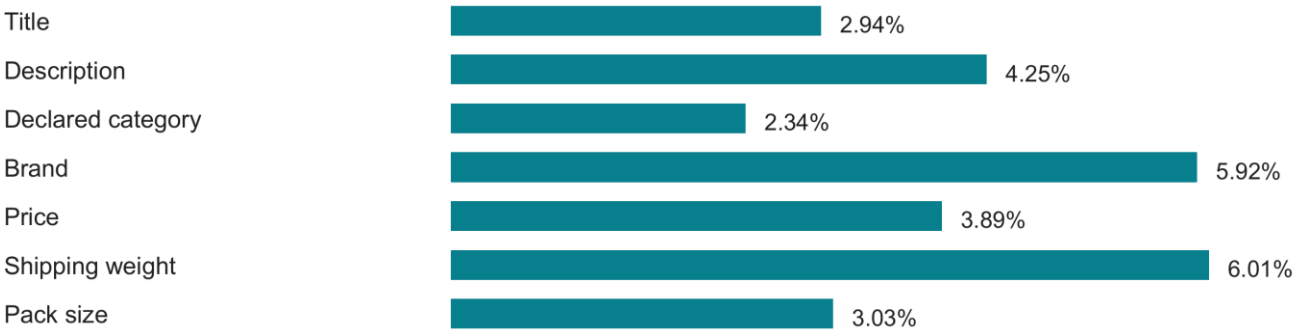


Figure 1 Missing values in the training data as a percentage of all 16,000 offers

Missingness is moderate, with the highest rates in brand and shipping weight at about 6%. There are 680 missing descriptions, but every one of those offers has a title. No training row is missing both text fields. This makes the title a useful fallback source of product information.

Quantities require context

Main-item quantities can be written as digits or words. A phrase such as Package contains 2 pcs describes the main quantity, while 3 pcs of free accessory pieces describes extras. The parser recognizes the complete phrase instead of treating the first number as the main count.

The combined title and description parser finds a main quantity for 15,990 offers, or 99.94%. Among 15,506 offers with both a parsed quantity and an observed pack size, 391 disagree. All 391 are positive in this dataset. This is a descriptive finding, not a universal rule for future data. The remaining 494 offers have an unknown comparison and are not counted as disagreements.

Redundant text and product names

All 15,320 nonempty descriptions contain a product phrase followed by a brand. Extracted brands agree with the structured brand in all 14,406 observed pairs. Text also contains the brand for 914 offers whose structured brand is missing. These checks show that useful information can remain in text when a structured field is absent.

Titles and descriptions often use different names for the same product. A dictionary learned from paired training text can connect these names. On held-out training sellers, this recovers product identity for 639 of the 680 offers with missing descriptions, or about 94.0%. Repeated text can also have different labels, so the method does not use text matches as a target lookup.

4 Approach

The pipeline first extracts product facts, then learns expected catalogue attributes, and finally uses their disagreements to predict corruption. It produces 38 model features. Offer and seller IDs are excluded from the model inputs.

Extracted facts and expected values

Text is normalized for case and spacing. Fixed patterns extract the product phrase, variant and main quantity. Title aliases fill unknown product identities when an unambiguous mapping is available. Unseen or ambiguous aliases remain unknown. Numeric inputs include log values, per-item ratios, missingness flags and quantity differences.

Reference models are fitted only on target-0 rows in the relevant training partition. Two ridge regressions predict log price and log shipping weight from product, variant, brand, condition, text quantity and promotion status. Ridge is a linear model with regularization to keep estimates stable. The observed price is excluded from its price predictor, and weight is excluded from its weight predictor.

Numeric residuals measure the difference between an observed log value and its prediction. Their sign shows the direction, and their absolute value shows the size. Product medians of log price and weight per declared item provide a second reference. These median residuals supplement the models that account for variant, condition and promotion context.

Category surprisal measures how unusual a product and declared category are together. It is the negative log of a smoothed category probability. Adding one to reference category counts prevents unseen combinations from producing infinite scores. Legitimate alternative categories remain possible, and missing product or category information gives an unknown score.

Validation without seller leakage

Five outer folds hold out entire sellers. Within each outer training partition, three inner seller folds generate the features used to train the classifier. Each row therefore receives reference features from models that excluded its seller. Alias dictionaries follow the same splitting policy. This prevents a row from helping create the reference features used to predict its own label.

The outer validation labels are used only to score predictions. They do not fit reference models, choose aliases or control early stopping. Each training row receives one out-of-fold prediction, meaning a prediction from a model that held out its seller.

Final classifier

CatBoost combines the extracted facts, original structured fields and reference deviations. The fixed settings are 450 trees, depth 5, learning rate 0.05 and regularization 5, with seed 42 and four CPU threads. The five fitted model and reference pairs are retained. After validation, their predicted probabilities are averaged with equal weight for final inference.

5 Results and Model Evaluation

Validation evaluates unseen sellers from the labelled training data. Mean fold ROC-AUC averages the five fold scores. Pooled OOF ROC-AUC combines all held-out predictions before scoring. The two values can differ slightly because they summarize predictions differently.

Measure	Result
Mean fold ROC-AUC	0.992409
Fold standard deviation	0.002046
Pooled OOF ROC-AUC	0.992251
OOF Average Precision	0.983699
OOF Brier score	0.013234

Outer fold	ROC-AUC	Average Precision
1	0.990801	0.981324
2	0.989762	0.979964
3	0.994064	0.986789
4	0.992094	0.982475
5	0.995323	0.987742

All fold AUC scores are high, with a range from 0.9898 to 0.9953. The results suggest that the consistency features separate most controlled errors across these seller groups. They do not establish the same performance on new product families or future text templates.

Description	Rows	OOF ROC-AUC	Average Precision
Present	15,320	0.9932	0.9860
Missing	680	0.9569	0.8764

Missing descriptions remain the weaker subgroup despite title recovery. The Brier score measures probability error, where lower is better. Calibration checks also show that some middle probability bins have few offers, so probability reliability should not be inferred from AUC alone.

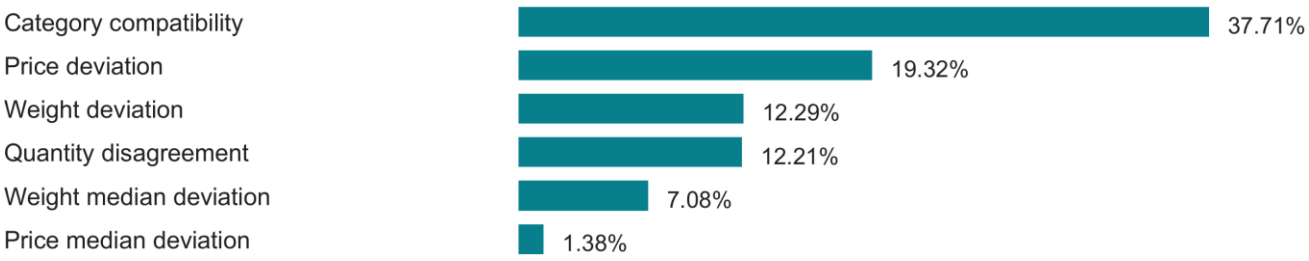


Figure 2 Top six normalized feature importances averaged across the five fold models

Category compatibility and price, weight and quantity deviations carry most of the model importance. Importance describes the fitted model; it is not a share of accuracy or proof of causation. These are development estimates because cross-validation was used during model development. No hidden-label test or leaderboard score is reported.

6 Summary

The project delivers a complete method for detecting controlled errors in catalogue offers. It uses information already present in the title, description and structured fields to estimate whether the offer is internally consistent. Learned references allow legitimate variation in categories, prices and weights, while quantity parsing separates main items from accessories.

The final ensemble achieves a mean seller-fold ROC-AUC of 0.9924 and OOF Average Precision of 0.9837. These results support the approach on the available training data. The strongest signals concern category compatibility and deviations from expected numerical attributes.

Limitations and further work

The parser depends on the supplied text grammar, and the alias dictionary can recognize only mappings learned from paired training text. Missing descriptions still reduce performance. Future evaluation should therefore include unfamiliar product names and templates, alongside a detailed review of remaining false positives and false negatives.

Further work should focus on conservative recovery of unknown aliases and better estimates of normal numeric variation for each product. Probability calibration and an operational alert threshold would need a separate evaluation. A proposed repair should remain alongside the original value until it has been verified.

Reproducibility and final output

The EDA and training notebooks contain their explanations, core functions and executed outputs. The EDA reads only the training file. The training notebook completes model fitting and validation before loading the prediction files for inference. It saves five model and reference pairs, OOF predictions, validation metrics, feature importance, calibration data and a model manifest.

The manifest records feature names, settings, package versions and file hashes. Reloading the saved ensemble reproduces the 10,000 exported probabilities. The submission format is id,target, with one probability per offer in the required order and no extra index column.

Project files

Deliverable	File
EDA notebook	notebooks/01_eda_report.ipynb
Training and submission notebook	notebooks/02_training_pipeline.ipynb
Model and validation artifacts	outputs/catalogue_submission/
Kaggle submission CSV	outputs/kaggle_submission/submission.csv

Sources

Data and method: the supplied training file and the two project notebooks. Numerical results: outputs/catalogue_submission/metrics.json. Model settings and input provenance: outputs/catalogue_submission/manifest.json. Feature usage: outputs/catalogue_submission/feature_importance.csv.